Line 4 — Monitoring & driving the crane with a TIA watch table

Goal: watch every `Line4Data` tag live, and — because the sim has no physical crane — **drive the limit switches and the start button by hand** so the `CraneSequencer` state machine walks through a batch exactly as it would on real hardware. This is the tool you use together with the app's **Send to PLC** button and the **Crane View** page.

> Prereqs: the program is downloaded and the CPU is in **RUN** (see `instructions.md` Parts 5–6). On **real hardware** you don't drive anything here — the switches come from wired `%I` inputs; you only *monitor*. See the last section.

Part 1 — Create the watch table

1. TIA project tree → **PLC_1** → **Watch and force tables** → **Add new watch table** (double-click).
2. It opens an empty table with columns **Name · Address · Display format · Monitor value · Modify value** and a checkbox column.

Part 2 — Add the tags

Click each **Name** cell and type these (autocomplete kicks in after `"Line4Data".`):

Handshake / status (read-only — the app + FB drive these):

```
"Line4Data".cmd_send
"Line4Data".task_count
"Line4Data".sts_state
"Line4Data".sts_busy
"Line4Data".sts_done
"Line4Data".sts_step
```

Limit switches + button (these are what YOU toggle in the sim):

```
"Line4Data".ls_at[0]
"Line4Data".ls_at[1]
"Line4Data".ls_at[2]
"Line4Data".ls_at[3]
"Line4Data".ls_at[4]
"Line4Data".ls_dip
"Line4Data".ls_up
"Line4Data".btn_start
```

A couple of task fields (to confirm the app's Send landed):

```
"Line4Data".tasks[1].station
"Line4Data".tasks[1].dwell_s
```

Part 3 — Go online + monitor

1. Click **"Go online"** (top toolbar). If asked: **PN/IE** → **Siemens PLCSIM Virtual Ethernet Adapter** → **Start search** → pick `CraneSim` → **Go online**. The frame turns orange.
2. In the watch-table toolbar click **"Monitor all"** (the **glasses** icon). The **Monitor value** column goes live.
3. At rest you should see `sts_state = 0`, all Bools `FALSE`, `task_count = 0`.

What each tag means

Tag	Dir	Meaning
`cmd_send`	app→PLC	TRUE = run this batch; cleared (FALSE) = acknowledge/idle
`task_count`	app→PLC	how many tasks are loaded (e.g. 3)
`sts_state`	read	state machine: 0 IDLE · 10 GO_LOAD · 20 LOWER · 30 WAIT_BTN · 40 LIFT · 50 GO_TANK · 60 DIP · 70 DWELL · 80 LIFT · 85 GO_PARK · 90 DONE
`sts_busy`	read	a batch is running
`sts_done`	read	batch finished (until `cmd_send` is cleared)
`sts_step`	read	current task number 1..count
`ls_at[s]`	input	limit switch — TRUE = crane is over station `s` (0=Parking, 1=Loading, 2=`1-1A`, 3=`1-1B`, 4=`1-2`, 5=`1-3`)
`ls_dip`	input	dip limit — TRUE = hoist fully lowered (in tank / loading)
`ls_up`	input	upper limit — TRUE = hoist fully raised
`btn_start`	input	operator start button (held 3 s at loading)

> **Station map:** `station = tank number + 1`. Parking = 0, Loading = 1, then tanks: `1-1A`=2, `1-1B`=3, `1-2`=4, `1-3`=5. The app computes this when it sends.

Part 4 — How to "flip a switch" (the key mechanic)

To simulate a limit switch or the button, use the **Modify value** column:

1. Type the value in the row's **Modify value** cell — for a **Bool** type `TRUE` or `FALSE`; for an **Int** type a number.
2. Right-click the row → **Modify** → **Modify now** (or click the toolbar button **"Modify all selected values once and now"** — the lightning-bolt icon).
3. The value is written once and **holds** — nothing else in the program drives these DB bits, so `btn_start = TRUE` stays TRUE until you set it back. That's how you "press and hold".

You can modify several rows at once: tick their checkboxes (or type in several **Modify value** cells), then **Modify** → **Modify now** applies them together.

Part 5 — Run a full batch to DONE (the demo)

Start the crane parked & raised, then send a recipe and drive it tank by tank.

0. Reset to a clean start (modify):

- `ls_at[0]` = TRUE (at Parking), all other `ls_at[..]` = FALSE
- `ls_up` = TRUE, `ls_dip` = FALSE, `btn_start` = FALSE
- `sts_state` should read **0**.

1. Arm it from the app — in the manager Dashboard pick `line4`, paste `1-1A=6;1-1B=3;1-2=5;`, **Generate tasks** → **Send to PLC**. Watch the table: `task_count`→3, `tasks[1].station`→2, `cmd_send`→TRUE, and `sts_state`→**10**, `sts_busy`→TRUE. (App shows "Tasks in progress".)

2. Drive the sequence — modify the switches as each state appears (watch `sts_state`):

`sts_state`	**What the crane is doing**	**Modify**
10 GO_LOAD	travelling to Loading	`ls_at[0]`=FALSE, `ls_at[1]`=TRUE
20 LOWER	lowering at Loading	`ls_up`=FALSE, `ls_dip`=TRUE
30 WAIT_BTN	waiting for operator	`btn_start`=TRUE, **wait ≥3 s**
40 LIFT	raising	`ls_dip`=FALSE, `ls_up`=TRUE, then `btn_start`=FALSE
50 GO_TANK (step 1)	to `1-1A` (station 2)	`ls_at[1]`=FALSE, `ls_at[2]`=TRUE
60 DIP	dipping	`ls_up`=FALSE, `ls_dip`=TRUE
70 DWELL	soaking ~6 s	*(wait — the `tasks[1].dwell_s` timer)*
80 LIFT	lifting out	`ls_dip`=FALSE, `ls_up`=TRUE → back to 50, `sts_step`→2
50 (step 2)	to `1-1B` (station 3)	`ls_at[2]`=FALSE, `ls_at[3]`=TRUE
60/70/80	dip, dwell ~3 s, lift	`ls_up`=FALSE,`ls_dip`=TRUE → wait → `ls_dip`=FALSE,`ls_up`=TRUE
50 (step 3)	to `1-2` (station 4)	`ls_at[3]`=FALSE, `ls_at[4]`=TRUE
60/70/80	dip, dwell ~5 s, lift	same pattern → last task → `sts_state`→85
85 GO_PARK	returning home	`ls_at[4]`=FALSE, `ls_at[0]`=TRUE
90 DONE	finished	— → `sts_done`=TRUE, `sts_busy`=FALSE

3. The app finishes the handshake — within ~1 s it toasts **"Tasks done"**, clears `cmd_send` (acknowledge), and re-enables **Send**. The PLC then returns `sts_state` to **0**, ready for the next recipe.

Open the **Crane View** page while you do this — the trolley glides between the boxes and the hoist dips as you flip the switches.

Tips & gotchas

- **Hold means hold.** `btn_start` must stay TRUE for the full 3 s (state 30) — a quick pulse won't trip the timer. Same idea: the dwell timers (state 70) run on their own; just wait.
- **Instant lower/lift is normal in the sim** if you leave `ls_dip`/`ls_up` both TRUE — states 20/40/60/80 only check one switch, so they pass through instantly. For a realistic dip, set `ls_dip`=FALSE and `ls_up`=TRUE *before* a tank, then swap them as it lowers.
- **Only one `ls_at` should be TRUE at a time** (the crane is over one station). Set the old one FALSE when you set the new one TRUE.
- **Stuck?** If `sts_state` won't advance, the state is waiting on a switch you haven't set — check the table above for that state's row. `sts_state` never leaving 0 after Send = no rising edge on `cmd_send` (it was already TRUE) or `task_count`=0.
- **Force vs Modify:** use **Modify** (once, holds) for this. The **Force** table permanently pins an input against the program — not needed here and easy to forget to unforce.

On real hardware (no manual toggling)

On a real PLC you **don't** modify these bits — you only monitor them. The limit switches and the button are wired to physical `%I` inputs and the motors to `%Q` outputs, mapped to the `Line4Data` I/O mirror in a small OB1 network (see `instructions.md` Part 9, "real hardware" note). Keep this watch table for **monitoring** the run and diagnosing faults; the crane drives itself. The app's **Send to PLC** and **Crane View** work identically — the only change is where the switch bits come from.